

8-BIT ALU DESIGN WITH 16 CHOICES IC

A Project Report

In the partial fulfillment of the award of the degree of

B. Tech

At

Ardent Computech Pvt. Ltd



Submitted by
Soumyadeep Mukherjee

Subhadip Hait

Suman Bera

Rup Mondal

Subhodeep Ghosh

Subhadeep Jana



JIS COLLEGE OF ENGINEERING



**ARDENT
COMPUTECH
PVT. LTD.**





Certificate from the Mentor

This is to certify that **Soumyadeep Mukherjee, Subhadip Hait, Suman Bera, Rup Mondal, Subhodeep Ghosh, Subhadeep Jana** has completed the project titled **8-bit ALU design with 16 choices IC** under my supervision during the period from **16/02/2026** to **27/02/2026** which is in partial fulfillment of requirements for the award of the **B.Tech** and submitted to Department **Electronics And Communication Engineering** of **JIS College Of Engineering**.

Leblanu Mondal

Signature of the Mentor

Date:27/02/2026



Acknowledgment

I take this opportunity to express my deep gratitude and sincerest thanks to my project mentor, **Debtanu Mondal** for giving the most valuable suggestions, helpful guidance, and encouragement in the execution of this project work.

I would like to give a special mention to my colleagues. Last but not least I am grateful to all the faculty members of the **Ardent Computech Pvt. Ltd.** for their support.



**ARDENT
COMPUTECH
PVT. LTD.**



8-BIT ALU DESIGN WITH 16 CHOICES IC

Abstract

This project presents the design of an 8-bit Arithmetic Logic Unit (ALU) capable of performing 16 different operations using select lines. The ALU executes arithmetic functions such as addition, subtraction, increment, and decrement, and logic operations like AND, OR, XOR, and NOT. The design is implemented using standard ICs including multiplexers and adder circuits. Four select lines control operation selection. The proposed ALU ensures high-speed performance, modular structure, and efficient hardware utilization suitable for digital system applications.

Table of Contents

Introduction	7
Objectives	7
Background	8
Understanding FPGA: Field-Programmable Gate Array.....	8
Introduction to Xilinx	10
Sample VHDL CODE	11
Tools / Environment Used	14
Design Methodology	14
[Put your design methodology here].....	14
VHDL Code.....	16
Simulations and Verification.....	19
Implementation on Spartan 7 FPGA.....	22
Spartan 7 FPGA development board.....	22
FPGA Configuration.....	23
Testing and Results.....	25
Conclusion.....	26
References.....	27

Introduction

An 8-bit Arithmetic Logic Unit (ALU) is a fundamental component of digital systems and microprocessors. It performs arithmetic and logical operations on 8-bit binary data. This design supports 16 different operations selected through four control lines. The ALU is implemented using standard integrated circuits such as adders, logic gates, and multiplexers. It plays a vital role in data processing, computation, and decision-making operations within digital electronic systems.

Objectives

The objective of this project is to design and implement an 8-bit Arithmetic Logic Unit capable of performing sixteen arithmetic and logical operations using select control lines. This project aims to develop a clear understanding of ALU architecture, function selection techniques, and IC-based hardware implementation. It also focuses on enhancing knowledge of digital logic design, circuit integration, and performance evaluation for efficient data processing in modern digital electronic systems.

Background

Understanding FPGA: Field-Programmable Gate Array

Field-Programmable Gate Arrays (FPGAs) are integrated circuits designed to be configured by the customer or designer after manufacturing—hence "field-programmable". They consist of an array of programmable logic blocks and a hierarchy of reconfigurable interconnects that allow these blocks to be wired together. This structure allows FPGAs to be programmed to perform complex digital computations and tasks.

Key Characteristics of FPGAs

1. **Reconfigurability**:- FPGAs can be reprogrammed to perform different functions or to optimize the performance of existing functions, making them highly versatile for various applications.
2. **Parallel Processing**:- Unlike traditional processors that execute instructions sequentially, FPGAs can perform multiple operations in parallel, significantly increasing computational efficiency for specific tasks.
3. **Low Latency**:- FPGAs can process data with very low latency because they can be tailored to the specific needs of an application, reducing the need for additional processing steps.
4. **Customizability**:- FPGAs allow for the implementation of custom hardware designs, which can be optimized for performance, power consumption, or both.

FPGA Architecture

1. **Configurable Logic Blocks (CLBs)**:- CLBs are the primary logic units in an FPGA. They contain look-up tables (LUTs), flip-flops, and multiplexers that can be configured to implement various logic functions.
2. **Interconnects**:- FPGAs use a complex matrix of interconnects to route signals between CLBs. These interconnects can be dynamically programmed to connect the desired logic blocks.
3. **I/O Blocks**:- Input/ Output blocks interface the FPGA with external devices. These blocks support various I/O standards and protocols, enabling the FPGA to communicate with other components in a system.
4. **Memory Blocks**:- FPGAs include dedicated memory blocks for temporary data storage, which can be used for tasks like buffering and caching.
5. **Clock Management**:- Clock management resources, such as phase-locked loops (PLLs) and clock dividers, are included to manage the timing of operations within the FPGA.

FPGA Development Process

1. **Design Entry**:- The design process starts with defining the logic and functionality, typically using Hardware Description Languages (HDLs) like VHDL or Verilog.
2. **Simulation**:- Before implementing the design on hardware, it is simulated to verify its correctness. Simulation tools allow designers to test and debug their logic.
3. **Synthesis**:- The HDL code is synthesized into a netlist, which describes the hardware components and their interconnections.

4. **Implementation:-** The netlist is mapped to the FPGA's resources (CLBs, interconnects, etc.). This step involves place-and-route algorithms to optimize the design's performance.
5. **Programming:-** The final step is to generate a bitstream file, which is used to program the FPGA. This file configures the FPGA to implement the desired logic.
6. **Verification and Testing:-** After programming the FPGA, the design is verified and tested in hardware to ensure it meets all functional and performance requirements.

Applications of FPGAs

1. **Telecommunications:-** FPGAs are used in network routers, switches, and base stations to perform high-speed data processing and protocol management.
2. **Consumer Electronics:-** In devices like digital cameras, smartphones, and gaming consoles, FPGAs are used for image processing, signal processing, and interfacing with other components.
3. **Automotive:-** FPGAs are employed in advanced driver-assistance systems (ADAS), infotainment systems, and electronic control units (ECUs).
4. **Industrial Automation:-** In industrial control systems, FPGAs provide the flexibility and performance required for real-time processing and control.
5. **Aerospace and Defense:-** FPGAs are used in radar systems, communication devices, and avionics due to their reliability and reconfigurability.
6. **Medical Devices:-** FPGAs enable high-performance processing in medical imaging systems, diagnostic equipment, and portable medical devices.

Advantages of FPGAs

- **Flexibility:-** Ability to reprogram for different applications and updates.
- **Performance** High-speed data processing with parallel execution.
- **Customization:** Tailored hardware solutions for specific needs.
- **Scalability:** Suitable for a wide range of applications from small devices to large systems.

Disadvantages of FPGAs

- **Cost:** Higher initial cost compared to ASICs for large-volume production.
- **Power Consumption:** Can consume more power than fixed-function devices.
- **Complexity:** Requires specialized knowledge for design and programming.

Introduction to VHDL Programming

VHDL (VHSIC (Very High Speed Integrated Circuits) hardware description language) is commonly used as a design-entry language for field-programmable gate arrays and application-specific integrated circuits in electronic design automation of digital circuits. VHDL was originally developed at the behest of the US Department of Defense in order to document the behavior of the ASICs that supplier companies were including in equipment. That is to say, VHDL was developed as an alternative to huge, complex manuals which were subject to implementation-specific details.

The idea of being able to simulate this documentation was so obviously attractive that logic simulators were developed that could read the VHDL files. The next step was the development of logic synthesis tools that read the VHDL, and output a definition of the physical implementation of the circuit. Modern synthesis tools can extract RAM, counter, and arithmetic

blocks out of the code, and implement them according to what the user specifies. Thus, the same VHDL code could be synthesized differently for lowest area, lowest power consumption, highest clock speed, or other requirements.

VHDL borrows heavily from the Ada programming language in both concepts (for example, the slice notation for indexing part of a one-dimensional array) and syntax. VHDL is not a case sensitive language. One can design hardware in a VHDL IDE (such as Xilinx or Quartus) to produce the RTL schematic of the desired circuit. After that, the generated schematic can be verified using simulation software (such as ModelSim) which shows the waveforms of inputs and outputs of the circuit after generating the appropriate test bench. To generate an appropriate test bench for a particular circuit or VHDL code, the inputs have to be defined correctly. For example, for clock input, a loop process or an iterative statement is required.

The key advantage of VHDL when used for systems design is that it allows the behavior of the required system to be described (modeled) and verified (simulated) before synthesis tools translate the design into real hardware (gates and wires). Another benefit is that VHDL allows the description of a concurrent system (many parts, each with its own sub-behavior, working together at the same time). VHDL is a Dataflow language, unlike procedural computing languages such as BASIC, C, and assembly code, which all run sequentially, one instruction at a time.

Introduction to Xilinx

Xilinx (an Electronics Company) is the world's largest supplier of programmable logic devices and the inventor of the field programmable gate array (FPGA). Today Xilinx is the worldwide leader of complete programmable logic solutions.

Xilinx has two main FPGA families – the high-performance video Virtex series and the high-volume Spartan series. Each model series has been released in multiple generations since its launch. The latest Virtex-6 and Spartan-6 FPGA families are said to consume 50 % less power , cost 20 % less , and have up to twice the logic capacity of previous generations of FPGAs. The Spartan series targets applications with low power footprint, extremes cost sensitivity and high volume e.g. displays, set top boxes wireless routers and other applications.

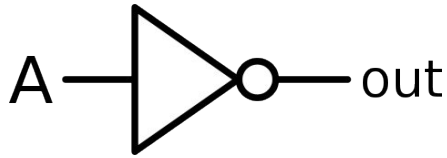
The ISE Design Suite is the central electronic design automation (EDA) product family sold by Xilinx .The ISE Design Suite features include design entry and synthesis supporting Verilog or VHDL ,place and route (PAR), completed verification and Chipscope pro tools , and creation of the bit files that are used to configure the chip .

Xilinx designs, develops and markets programmable logic products including integrated circuits (ICs), software design tools and services . Xilinx sells both FPGAs and CPLDs (Complex Programmable Logic Devices) for electronic equipment manufacturers.

In this project we used the Xilinx Vivado 2023.2 version to complete our project.

Sample VHDL CODE

1-BIT NOT GATE



TRUTH TABLE

Input	Output
A	Y
0	1
1	0

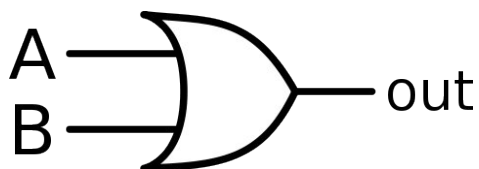
VHDL CODE

```
library ieee; use
ieee.std_logic_1164.all;      entity
not_gate is port (a:in std_logic;
c:out std_logic);      end not_gate;
architecture Behavioral of
not_gate is begin
C<= not A;      end Behavioral
```

TEST BENCH WAVEFORM



1-BIT OR GATE



TRUTH TABLE

Inputs		Output
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

VHDL CODE

```
library      ieee;      use
ieee.std_logic_1164.all;
```

*entity or_gate is port (a:in
std_logic; b:in std_logic;*

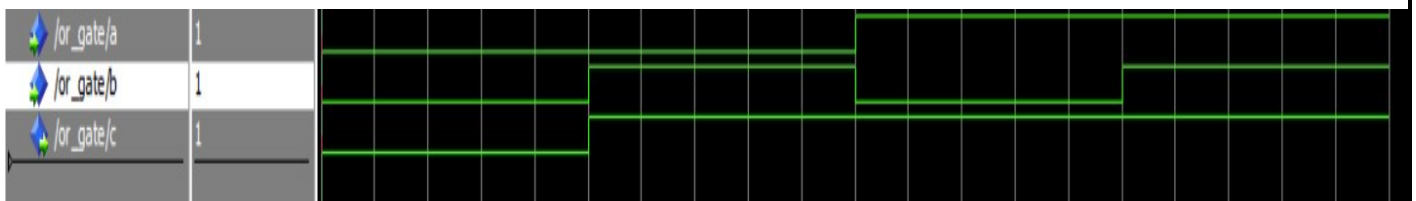
```

c:out std_logic);
end or_gate;

architecture Behavioral of or_gate is
begin
C<= A or B;
end Behavioral;

```

TEST BENCH WAVEFORM



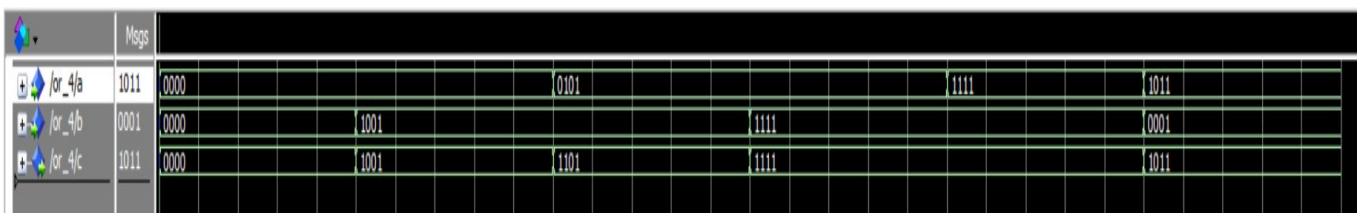
4-BIT OR GATE VHDL CODE

```

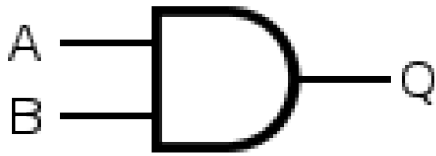
library ieee;
use ieee.std_logic_1164.all;
entity or_4 is
port (a:in std_logic_vector(0 to 3);
b:in std_logic_vector(0 to 3);
c:out std_logic_vector(0 to 3));
end or_4;
architecture Behavioral of or_4 is
begin
c<= A or B;
end Behavioral;

```

TEST BENCH WAVEFORM



1-BIT AND GATE



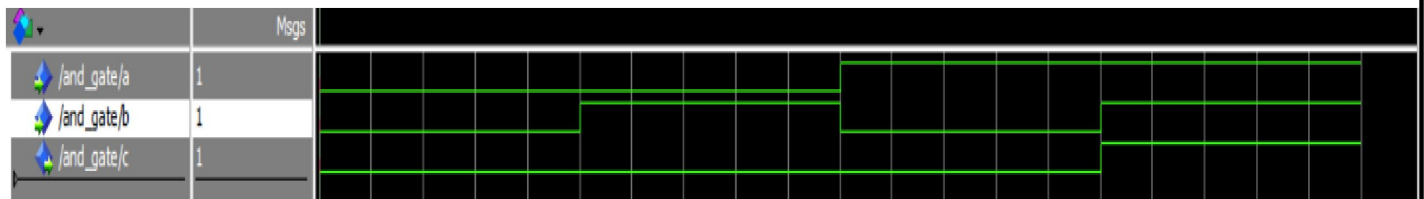
VHDL CODE

```
library ieee;
use ieee.std_logic_1164.all;
entity and_gate is
port (a:in std_logic;
b:in std_logic;
c:out std_logic);
end and_gate;
architecture Behavioral of and_gate is
begin
C<= A and B;
end Behavioral;
```

TRUTH TABLE

Inputs		Output
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

TEST BENCH WAVEFORM



4-BIT AND GATE

VHDL CODE

```
library ieee;
use ieee.std_logic_1164.all;
entity and_4 is
port (a:in std_logic_vector(0 to 3);
b:in std_logic_vector(0 to 3);
c:out std_logic_vector(0 to 3));
end and_4;
architecture Behavioral of and_4 is
begin
c<= A and B;
end Behavioral;
```

TEST BENCH WAVEFORM

Constraint File->The Constraint File is used to assign the logical ports defined in the VHDL design to the physical pins of the FPGA kit. Since every FPGA board has a unique pin configuration, the constraint file plays a crucial role in ensuring proper hardware implementation.

In this file, the input switches and output display pins of the FPGA kit are mapped to the corresponding ports defined in the design file (such as '**bin**' for input and '**seg**' for output). This mapping ensures that when a switch is toggled on the FPGA board, the correct signal is received by the design, and the appropriate output is displayed on the 7-segment display.

It is important to note that no two FPGA kits have identical pin configurations. Therefore, the constraint file must strictly follow the pin details provided in the FPGA kit's user manual. If the port assignments in the constraint file do not match the actual hardware pin configuration, the design will not function correctly, and it may become difficult to identify the issue.

Thus, the constraint file serves as a bridge between the logical design and the physical FPGA hardware, ensuring accurate and effective execution of the project.

VHDL Code

```
library IEEE; use
IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity alu_8bit is
Port (
A : in STD_LOGIC_VECTOR(0 to 7);
B : in STD_LOGIC_VECTOR(0 to 7);
S : in STD_LOGIC_VECTOR(0 to 3);
Y : out STD_LOGIC_VECTOR(0 to 15)
);
end alu_8bit;

architecture Behavioral of alu_8bit is
begin

process(A, B, S)

variable A_u : unsigned(0 to 7);
variable B_u : unsigned(0 to 7);
variable res_u : unsigned(0 to 7);
variable mult : unsigned(0 to 15);

begin

A_u := unsigned(A);
B_u := unsigned(B);

case S is

when "0000" =>
res_u := A_u + B_u;

when "0001" =>
res_u := A_u - B_u;

when "0010" =>
mult := A_u * B_u;
res_u := mult(0 to 7);

when "0011" => if
B_u /= 0 then res_u :=
A_u / B_u; else
res_u := (others =>
'0'); end if;
```

when "0100" =>
res_u := A_u and B_u;


```

when "0101" =>
    res_u := A_u or B_u;

when "0110" =>
    res_u := A_u xor B_u;

when "0111" =>
    res_u := not A_u;

when "1000" =>
res_u := not (A_u and B_u);

when "1001" =>
res_u := not (A_u or B_u);

when "1010" =>
res_u := not (A_u xor B_u);

when "1011" =>
    res_u := A_u + 1;

when "1100" =>
    res_u := A_u - 1;

when "1101" =>
res_u := shift_left(A_u, 1);

when "1110" =>
res_u := shift_right(A_u, 1);

when "1111" =>
    res_u := B_u;

when      others      =>
res_u := (others => '0');

    end case;

Y <= std_logic_vector(res_u);

end process;

```

end Behavioral;

Simulation and Verification

The VHDL code is first simulated using simulation tool such of Vivado to verify the correctness of the logic. The simulation involves applying different inputs and observing the corresponding outputs on the Spartan 7 FPGA Development Board.

VHDL TESTBENCH FILE CODE

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL; use
IEEE.NUMERIC_STD.ALL;

entity alu_8bit_tb is end
alu_8bit_tb;

architecture Behavioral of alu_8bit_tb is

    component alu_8bit
        Port (
            A : in  STD_LOGIC_VECTOR(0 to 7);
            B : in  STD_LOGIC_VECTOR(0 to 7);
            S : in  STD_LOGIC_VECTOR(0 to 3);
            Y : out STD_LOGIC_VECTOR(0 to 15)
        );
    end component;

    signal A : STD_LOGIC_VECTOR(0 to 7);
    signal B : STD_LOGIC_VECTOR(0 to 7);
    signal S : STD_LOGIC_VECTOR(0 to 3);
    signal Y : STD_LOGIC_VECTOR(0 to 15);

begin
    uut:
    alu_8bit
```

```
port map (  
  A => A,  
  B => B,  
    S => S,  
    Y => Y  
);
```

```
process  
begin  
  A <= "00001010";  
  B <= "00000011";  
  
  S <= "0000"; wait for 10 ns;  
  
  S <= "0001"; wait for 10 ns;  
  
  S <= "0010"; wait for 10 ns;  
  
  S <= "0011"; wait for 10 ns;  
  
  S <= "0100"; wait for 10 ns;  
  
  S <= "0101"; wait for 10 ns;  
  
  S <= "0110"; wait for 10 ns;  
  
  S <= "0111"; wait for 10 ns;  
  
  S <= "1000"; wait for 10 ns;
```


S <= "1001"; wait for 10 ns;

S <= "1010"; wait for 10 ns;

S <= "1011"; wait for 10 ns;

S <= "1100"; wait for 10 ns;

S <= "1101"; wait for 10 ns;

S <= "1110"; wait for 10 ns;

S <= "1111"; wait for 10 ns;

wait;

end process;

end Behavioral;

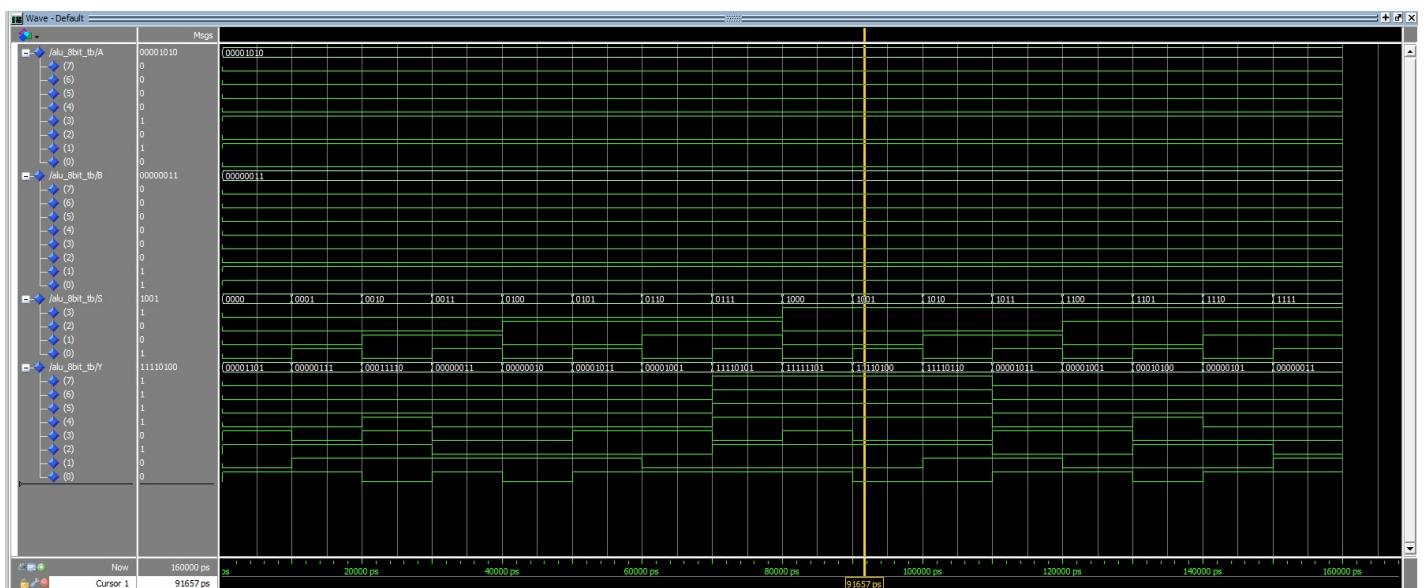
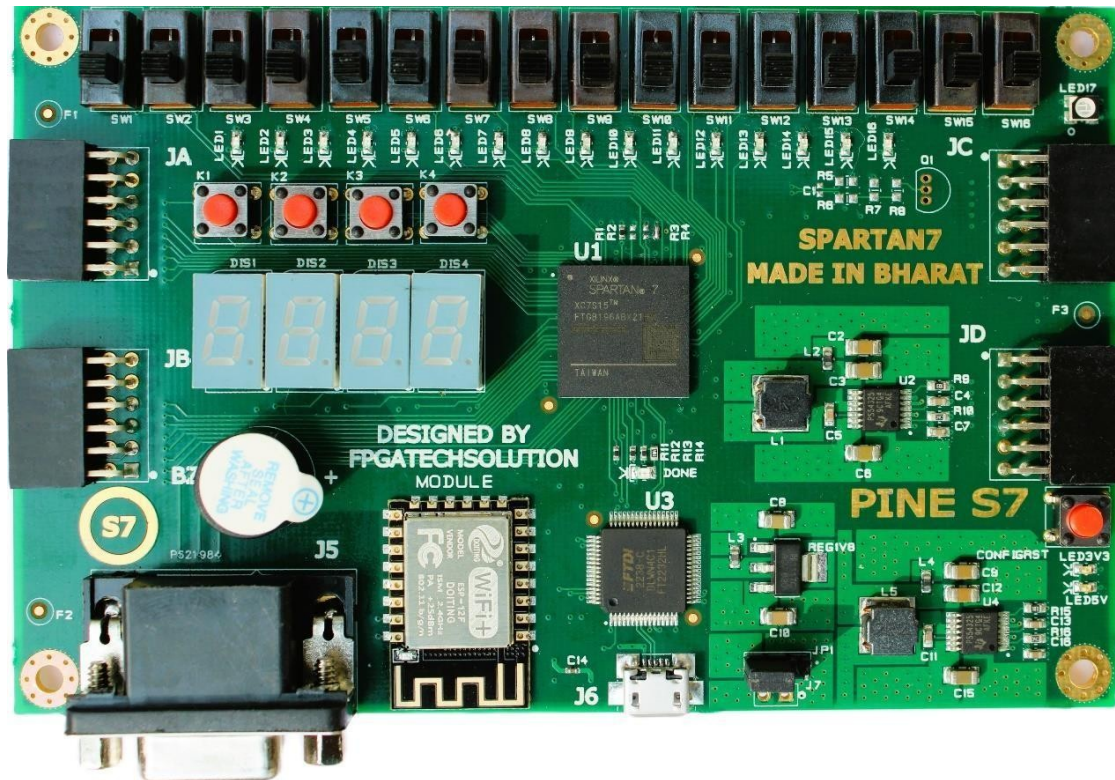


Figure: Test Bench Wave Form

Implementation on Spartan 7 FPGA

Spartan 7 FPGA development board



PINE S7 is an easy to use FPGA Development board featuring SPARTAN7 FPGA (XC7S15FTGB196-1), with FTDI's FT2232H Dual-Channel USB device, which support Xilinx Vivado, SDK & Vitis to program FPGA. It is specially designed for the development and integration of FPGA based accelerated features to other designs. This board works with USB power however provision is given for external 5V power supply

Features

- 16 user switches
- 16 user LEDs
- 4 user pushbuttons
- 100MHz clock
- 4-digit 7-segment display
- Four Pmod ports
- ESP-12E WiFi
- Buzzer
- RGB LED
- VGA output
- USB-UART Bridge
- Serial Flash
- USB-JTAG port for FPGA programming and communication (support Vivado)

FPGA Configuration

The synthesized design is implemented on the FPGA using Vivado Design Suite. The seven-segment display is connected to the appropriate GPIO pins of the Spartan 7 FPGA development board. Pin assignments are made according to the board's pinout configuration.

Constraint File (.XDC file)

```
#switch set_property -dict { PACKAGE_PIN A5   IOSTANDARD LVCMOS33 } [get_ports
{ A[0] }]; set_property -dict { PACKAGE_PIN B6   IOSTANDARD LVCMOS33 } [get_ports
{ A[1] }]; set_property -dict { PACKAGE_PIN F4   IOSTANDARD LVCMOS33 } [get_ports
{ A[2] }]; set_property -dict { PACKAGE_PIN C3   IOSTANDARD LVCMOS33 } [get_ports
{ A[3] }]; set_property -dict { PACKAGE_PIN C4   IOSTANDARD LVCMOS33 } [get_ports
{ A[4] }]; set_property -dict { PACKAGE_PIN A10  IOSTANDARD LVCMOS33 }
[get_ports { A[5] }]; set_property -dict { PACKAGE_PIN B13  IOSTANDARD
LVCMOS33 } [get_ports { A[6] }]; set_property -dict { PACKAGE_PIN C14
IOSTANDARD LVCMOS33 } [get_ports { A[7] }];
```

```
set_property -dict { PACKAGE_PIN D13  IOSTANDARD LVCMOS33 } [get_ports { B[0] }];
set_property -dict { PACKAGE_PIN F14  IOSTANDARD LVCMOS33 } [get_ports { B[1] }];
set_property -dict { PACKAGE_PIN G14  IOSTANDARD LVCMOS33 } [get_ports { B[2] }];
set_property -dict { PACKAGE_PIN H13  IOSTANDARD LVCMOS33 } [get_ports { B[3] }];
set_property -dict { PACKAGE_PIN D12  IOSTANDARD LVCMOS33 } [get_ports { B[4] }];
set_property -dict { PACKAGE_PIN E12  IOSTANDARD LVCMOS33 } [get_ports { B[5] }];
set_property -dict { PACKAGE_PIN F11  IOSTANDARD LVCMOS33 } [get_ports { B[6] }];
set_property -dict { PACKAGE_PIN H11  IOSTANDARD LVCMOS33 } [get_ports { B[7] }];
```

```
#buttons set_property -dict { PACKAGE_PIN B5   IOSTANDARD LVCMOS33 }
[get_ports { S[0] }]; set_property -dict { PACKAGE_PIN A4   IOSTANDARD
LVCMOS33 } [get_ports { S[1] }]; set_property -dict { PACKAGE_PIN A3
IOSTANDARD LVCMOS33 } [get_ports { S[2] }]; set_property -dict { PACKAGE_PIN B3
IOSTANDARD LVCMOS33 } [get_ports { S[3] }];
```

```
#led set_property -dict { PACKAGE_PIN F3   IOSTANDARD LVCMOS33 } [get_ports
{Y[0]}}; set_property -dict { PACKAGE_PIN E4   IOSTANDARD LVCMOS33 } [get_ports
{Y[1]}}; set_property -dict { PACKAGE_PIN D3   IOSTANDARD LVCMOS33 } [get_ports
{Y[2]}}; set_property -dict { PACKAGE_PIN D4   IOSTANDARD LVCMOS33 } [get_ports
{Y[3]}}; set_property -dict { PACKAGE_PIN C5   IOSTANDARD LVCMOS33 } [get_ports
{Y[4]}}; set_property -dict { PACKAGE_PIN A12  IOSTANDARD LVCMOS33 } [get_ports
{Y[5]}}; set_property -dict { PACKAGE_PIN A13  IOSTANDARD LVCMOS33 } [get_ports
{Y[6]}}; set_property -dict { PACKAGE_PIN B14  IOSTANDARD LVCMOS33 } [get_ports
{Y[7]}}; set_property -dict { PACKAGE_PIN D14  IOSTANDARD LVCMOS33 } [get_ports
{Y[8]}}; set_property -dict { PACKAGE_PIN E13  IOSTANDARD LVCMOS33 } [get_ports
{Y[9]}}; set_property -dict { PACKAGE_PIN F13  IOSTANDARD LVCMOS33 } [get_ports
{Y[10]}}; set_property -dict { PACKAGE_PIN H14 IOSTANDARD LVCMOS33 } [get_ports
{Y[11]}}; set_property -dict { PACKAGE_PIN J14 IOSTANDARD LVCMOS33 } [get_ports
{Y[12]}}; set_property -dict { PACKAGE_PIN C12 IOSTANDARD LVCMOS33 } [get_ports
{Y[13]}}; set_property -dict { PACKAGE_PIN E11 IOSTANDARD LVCMOS33 } [get_ports
{Y[14]}}; set_property -dict { PACKAGE_PIN F12 IOSTANDARD LVCMOS33 } [get_ports
{Y[15]}};
```

Testing and Results

The FPGA is configured with the synthesized design, and various BCH inputs are applied to verify the numerical representation on the seven-segment display. The testing process involves:

- Configuring the FPGA with the synthesized design.

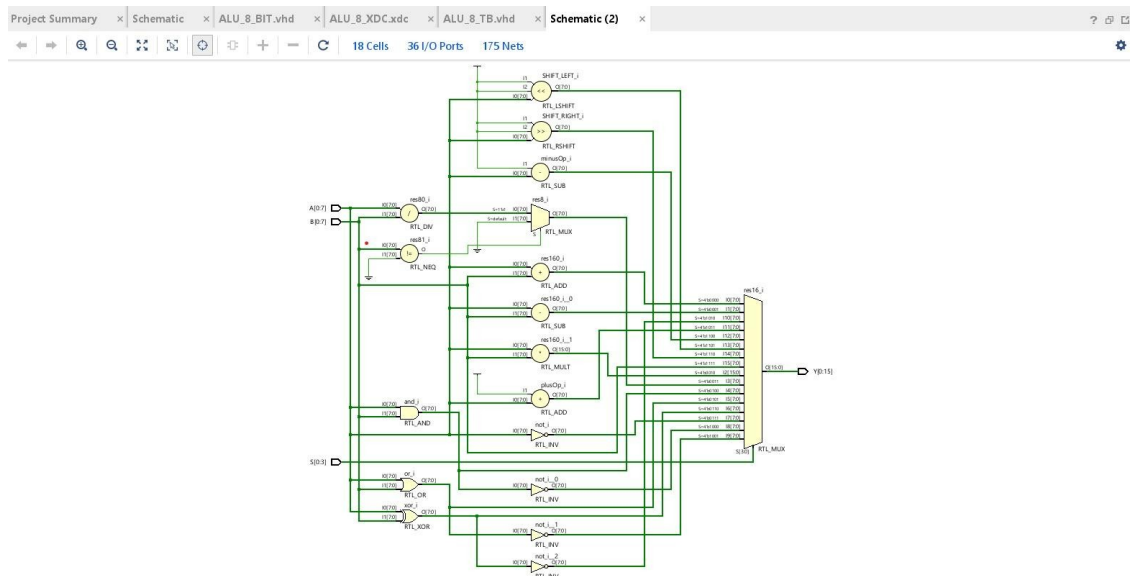


Figure: Synthesized design

- Applying test inputs using switches or buttons on the FPGA board.
- Observing and verifying the outputs on the seven-segment display.

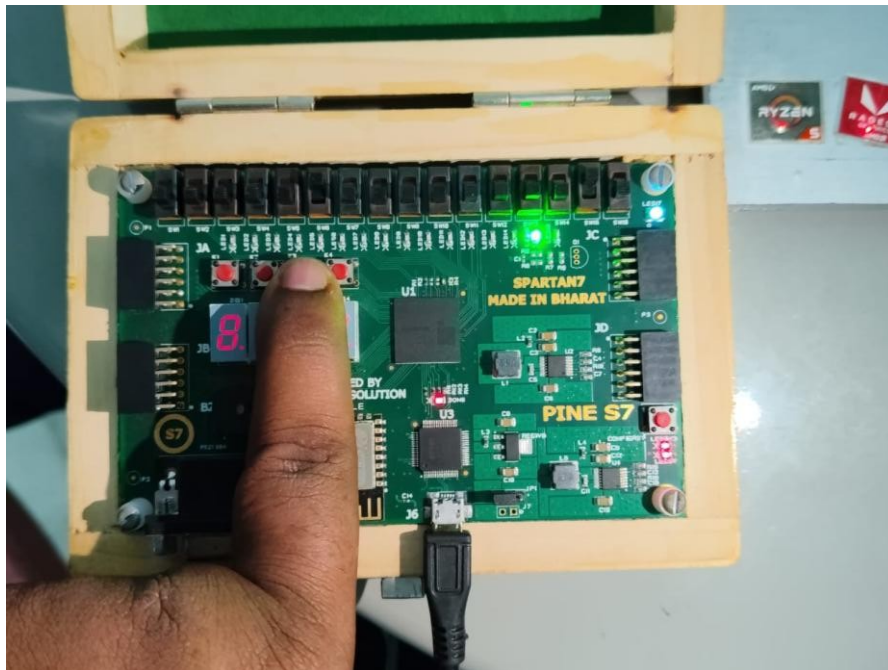


Figure: Output image of the FPGA Board

Conclusion

FPGAs are powerful tools in the field of digital design, offering unparalleled flexibility and performance for a wide range of applications. Their ability to be reprogrammed and customized

makes them ideal for industries where rapid prototyping and iteration are crucial. As technology advances, FPGAs continue to play a vital role in the development of modern electronic systems.

The designed 8-bit ALU successfully performs sixteen arithmetic and logical operations using select control lines. The implementation using standard ICs provides reliable performance and a modular structure. This project strengthens understanding of ALU architecture, operation selection techniques, and digital circuit integration. The developed system demonstrates efficient data processing and can be effectively used in microprocessorbased and other digital electronic systems.

References

1. Short, K. L. (2009). *VHDL for Engineers*. Prentice Hall.
2. Chang, K. C. (1997). *Digital Design and Modeling with VHDL and Synthesis*. IEEE Computer Society Press.
3. Perry, D. L. (2002). *VHDL: Programming by Example*. McGraw-Hill.
4. Ashenden, P. J. (2008). *The Designer's Guide to VHDL*. Morgan Kaufmann.
5. VHDLwhiz. (n.d.). *VHDL Reference Guide*. Retrieved from <https://vhdlwhiz.com>
6. Digi-Key Electronics. (n.d.). Introduction to VHDL. Retrieved from <https://www.digikey.com/en/resources/design-tools/technical-reference/vhdl>
7. Kumar, R., & Das, S. (2015). VHDL in Digital System Design and Synthesis. *International Journal of Electronics and Communication Engineering*, 7(2), 145-152.
8. Smith, J., Johnson, M., & Brown, A. (2018). A Review of VHDL-Based Digital Design Techniques. *IEEE Transactions on Education*, 61(3), 234-245.
9. Xilinx Inc. (2023). Vivado Design Suite User Guide. Retrieved from <https://www.xilinx.com/support/documentation-navigation/design-hubs.html>.
10. Xilinx Inc. (2023). Spartan-7 FPGA Family Data Sheet. Retrieved from <https://www.xilinx.com/products/silicon-devices/fpga/spartan-7.html>.